FOR COMPLETE BEGINNERS

Same call. Different result.

Complete Modern Java, episode 29: polymorphism, in Java and TypeScript

> When checkout keeps asking what kind of payment it has, let each payment object answer instead.

An if statement is not wrong; for two fixed types it may be the simplest thing. But every new type means editing it.

With a shared type, the call stays the same and each object supplies its own behaviour.

In this episode

- A string if chain
- totalFor(PaymentMethod)
- Adding a type
- One variable, two objects
- What polymorphism means
- TypeScript, the same way

Coming next

- Records (episode 30)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

StringTill: card, cash, voucher	`1020, 1000, unknown payment type`
Till: card, cash	`1020, 1000`
Till: a VoucherPayment	`1050, totalFor unchanged`
one variable: a card, then cash	`1020, then 1000`
till.ts	`1020, 1000, 1050`

Every row was run, not predicted.

02 Asking what kind of payment it is

Fine for a small fixed set; the method has to be edited for every new payment type.

StringTill.java

```
int totalFor(String type, int priceCents) {
    if (type.equals("card")) {
        return priceCents + priceCents * 2 / 100;
    }
    if (type.equals("cash")) {
        return priceCents;
    }
    throw new IllegalArgumentException("unknown payment type: " + type);
}
```

> card 1020, cash 1000, then unknown payment type: voucher.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 Letting the object answer

totalFor never asks what kind of payment it was given. A new VoucherPayment class is created and passed in; the calculation does not change.

Every payment uses the same inherited totalCents; inside it, Java calls the feeCents of the actual object.

Till.java

```
int totalFor(PaymentMethod method, int priceCents) {
    return method.totalCents(priceCents);
}

PaymentMethod method = new CardPayment();
IO.println(method.totalCents(1000));   // 1020
method = new CashPayment();
IO.println(method.totalCents(1000));   // 1000
```

> Card 1020, cash 1000, voucher 1050; one variable printed 1020, then 1000.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript

The same abstract class, three payment types and a totalFor function give the same three answers.

till.ts

```
function totalFor(method: PaymentMethod, priceCents: number): number {
    return method.totalCents(priceCents);
}
console.log(totalFor(new VoucherPayment(), 1000));   // 1050
```

> tsc quiet; 1020, 1000, 1050.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Polymorphism, in one look

1. Same call, through a shared type.
2. The object decides which version runs.
3. An if is not wrong; a growing one is a hint.
4. New type: new class, the checkout unchanged.
5. Challenge: a fourth payment type, totalFor untouched.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.